Replicated Editing: Collaborative Text via Multi-Paxos and Yjs

Vihaan Gupta
Harvard College
vihaangupta@college.harvard.edu

1 Introduction

Collaborative text editing where multiple users edit the same document simultaneously and see each other's changes in real time is one of the most demanding consistency problems in distributed systems. Systems like Google Docs must guarantee that concurrent edits from different users converge to the same document state, even under network delays and partial failures. Achieving this requires solving two independent problems: a convergence problem (i.e how do concurrent edits merge without losing either user's work?) and a fault-tolerance problem (i.e how does the system survive server failures without losing committed edits?).

This paper describes a collaborative editor built on top of a Multi-Paxos replicated log combined with Yjs, a CRDT library, to handle the convergence problem. Using Yjs, every keystroke is encoded as an opaque binary update blob that is sent to a server via a webstocket connection. We then use Multi-Paxos on the backend to handle fault-tolerance where blobs from the client are replicated across a cluster of servers before being broadcast back to clients. Finally, Yjs on the frontend ingests the blobs broadcasted from the server to ensure convergence for all clients connected to a single text window.

We evaluate our system across three metrics. First, we measure end-to-end write latency across four configurations: a single-server no-consensus baseline and Paxos clusters of 3, 5, and 7 replicas, to quantify the cost of introducing consensus and whether that cost grows with replica count. Second, we measure failover delay and blob loss after abruptly killing the server, to verify that Paxos configurations recover within a reasonable time window while the baseline does not. Third, we stress-test correctness under concurrency by connecting 25 simultaneous browser clients and issuing repeated waves of concurrent edits, to determine whether the system converges correctly under sustained load.

2 Related Work

2.1 Consensus and replication

Our system builds directly on Lamport's Multi-Paxos protocol [3], which extends basic Paxos to efficiently replicate a sequence of log entries across a cluster of servers. Instead of traditional Paxos, where all servers must converge to a single value, Multi-Paxos runs Paxos as a subroutine to achieve convergence over a log of operations. Each log is assigned a slot number which provides a deterministic ordering for the logs; this is exactly what we leverage in ensuring consistency within our model.

While Lamport's presentation focuses on the theoretical properties of the protocol, Chandra et al. [1] describe the significant engineering challenges that arise when deploying Paxos in a real production system: including leader election subtleties, disk corruption, and the gap between the protocol's idealized model and real hardware. While implementing our system, we encountered similar practical challenges that are not well reflected in theory: bridging the simulated network with real browser WebSocket, ensuring late-joining clients received a consistent document snapshot, and preventing echo loops where a client would apply its own update twice after the server broadcast it back. Each of these is discussed in detail in Section 4.

2.2 Collaborative editing

The problem of merging concurrent edits from multiple users has a long history. Nichols et al.'s Jupiter system [4] was one of the earliest deployed collaborative editors, and its central insight that a server-mediated total order over operations simplifies convergence is what inspired the client-server paradigmn for collaborative editing. Jupiter used Operational Transformation (OT) to reconcile concurrent edits, where each operation is transformed relative to concurrent operations before being applied, so that all clients converge to the same state regardless of the order in which they receive updates.

Because we had already committed to a Paxos replicated log as a source of truth for ordering, it took care the core problem that makes OT viable in the first place. Rather than needing a merge strategy capable of reconciling edits arriving in arbitrary order, we only needed a compact and correct encoding of document changes. This is where CRDTs, and specifically Yjs, comes in. Yjs provides a blob encoding with strong convergence guarantees, which fit perfectly within the existing architecture.

2.3 Yjs

Conflict-free Replicated Data Types (CRDTs), formalized by Shapiro et al. [5], are data structures designed so that concurrent updates can always be merged deterministically without coordination. The key property that enables this is commutativity: applying update A then B produces the same result as

applying B then A, meaning nodes in a peer-to-peer system can exchange updates in any order and still converge without a central authority imposing an ordering.

Yjs [2] is a operation-based CRDT library for collaborative text editing. Rather than tracking character positions, Yjs assigns every character a globally unique identifier, so concurrent insertions at the same position can always be resolved deterministically by comparing identifiers. Each change is encoded as a compact binary blob that can be applied in any order and still converge to the same document state. We use Yjs for its blob encoding and local document state management, while the Paxos backend takes care of total operation ordering. These are two fundamentally different problems, as we will discuss in section 3.2.

3 System Design

3.1 System Overview

Our system is a fault-tolerant collaborative text editor composed of three layers. The frontend is a single static HTML page running a Yjs CRDT document in the browser, connected to the server via a persistent WebSocket connection. The networking layer implements RFC 6455 WebSocket framing, manages connected clients, and bridges browser traffic to the backend through a shared client_bridge. The backend is a Multi-Paxos replicated log that imposes a total order over all edits before broadcasting them back to clients. Once committed, each blob is broadcast as a binary WebSocket frame to all connected clients and appended to an in-memory committed_blob_log. Late-joining clients receive a full replay of this log in commit order before being admitted to the live broadcast hub, ensuring no edit is missed regardless of when a client connects.

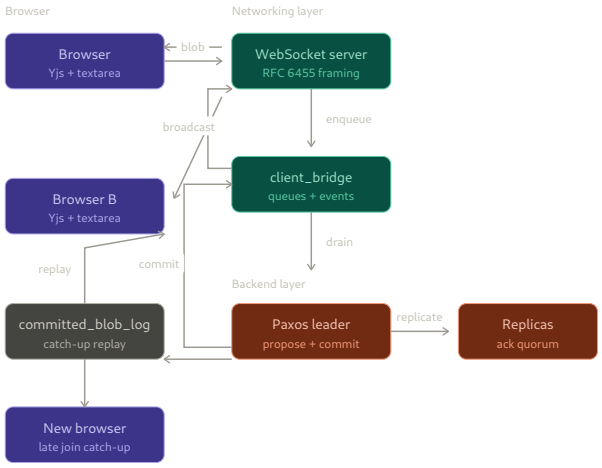

Figure 1: System architecture dataflow

Our system provides two fundamental guarantees: consistency (ensured by the Paxos log imposing a total order over all edits) and fault tolerance (ensured by Multi-Paxos replicating every committed blob across a quorum of replicas).

3.2 Consistency through Ordering and Convergence

Consistency in a collaborative editor requires solving two independent subproblems: ensuring all clients agree on the same order of edits, and ensuring concurrent edits from different clients can always be merged into a coherent document state. Most systems must solve these simultaneously since without a central authority, clients must rely entirely on the CRDT's merge logic to reconcile edits that arrive in different orders at different replicas.

Our design separates these concerns and solves each independently, then combines them to achieve full consistency. The Paxos log solves the ordering problem: every blob is committed to a totally ordered log before any client applies it, so all clients receive edits in exactly the same sequence broadcasted by the Paxos backend. Yjs solves the convergence problem: each edit is encoded as a binary blob with stable character identities that can be applied deterministically to a local document state. Together, the two guarantees compose cleanly: all client recieve the same log of operations from the backend, and each client that receives the same ordered sequence of Yjs blobs as any other client will always arrive at the same document state.

3.3 Fault Tolerance Model

Our system inherits the fault tolerance properties of Multi-Paxos. A cluster of $2f + 1$ replicas can tolerate up to f simultaneous failures while continuing to commit new blobs, since a quorum of $f + 1$ replicas is sufficient to make progress. In our evaluation we test configurations of 3, 5, and 7 replicas, tolerating 1, 2, and 3 failures respectively.

Leader failure is detected via timeout. If a follower replica does not hear from the leader within a configurable window, it assumes the leader has failed and initiates re-election by incrementing the round number and running Paxos phase 1. During the re-election window, from the moment the leader fails to the moment a new leader has been elected and committed its first entry, the system is temporarily unavailable: blobs arriving from browsers are queued in `client_bridge` but not proposed. Once a new leader is elected, it drains the queue and resumes normal operation. The length of this unavailability window is the primary cost of fault tolerance and is one of the metrics we measure in our evaluation.

3.4 The Bridge Abstraction

A practical challenge in our design is that the Paxos layer and the WebSocket layer operate in fundamentally different execution contexts. The Paxos replication runs on netsim, an in-process simulated network using typed C++ coroutines and channels. Instead of using real TCP connections, netsim models message delivery as coroutine delays via `co_await cot::after(link_delay_)`. The WebSocket server, by contrast, handles real OS-level TCP connections from browsers: it calls `bind`, `listen`, and `accept` on

actual file descriptors, reads and writes real bytes, and must interoperate with a standard browser WebSocket client speaking RFC 6455. These two models needed to communicate without either being aware of the other's I/O model. We solve this with `client_bridge`, a lightweight shared structure that sits at the boundary between the two layers.

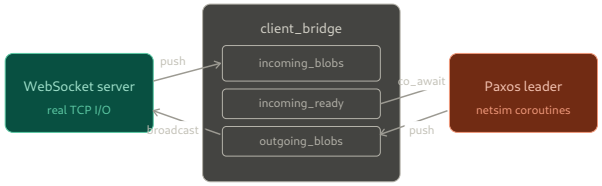

Figure 2: client_bridge dataflow

The WebSocket server pushes incoming browser blobs into `incoming_blobs` and triggers `incoming_ready`. This decouples the two models cleanly: the WebSocket server does not call into Paxos directly, it simply deposits a blob and signals that work is available. On the other side, the Paxos leader `co_awaits` on `incoming_ready`, then drains the queue and proposes each blob as a log entry through the normal Multi-Paxos replication path.

The same pattern applies in reverse. Once the leader commits a blob, it pushes it into `outgoing_blobs` and triggers `outgoing_ready`, waking a broadcast coroutine that sends each blob as a binary WebSocket frame to every connected client. The result is that the netsim coroutine world and the real TCP world communicate exclusively through this shared queue-and-event interface, with neither layer ever directly calling into the other.

4 Implementation

4.1 Frontend

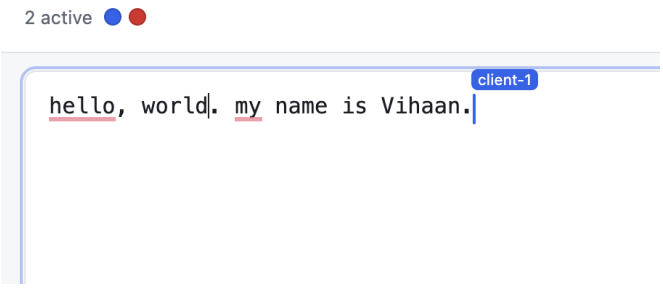

Figure 3: Text Editor UI

The frontend is a single-page web application with plain HTML file that loads Yjs from a CDN and opens a WebSocket connection to the server. The collaborative document model is a `Y.Doc` with a single `Y.Text("content")` shared type, which represents the document as a sequence of characters with stable unique identities. This shared type is bound to a plain `<textarea>` as the editing surface.

When the user types, the browser fires an `input` event. A diff helper (`computeTextDelta`) compares the new textarea value against the previous one to identify what changed (an insertion, deletion, or replacement) and translates it into the corresponding Yjs operation wrapped in a `ydoc.transact()` call. Yjs then encodes that operation as a compact binary blob and this blob is then sent to the server over the WebSocket connection.

In the other direction, when the server broadcasts a committed blob back, the frontend receives it and passes it to `Y.applyUpdate()`, which applies the change to the local `Y.Doc`. The updated document state is then reflected back into the textarea via `syncYTextToTextarea()`.

Client presence and cursor metadata are handled over the same WebSocket channel via JSON control messages distinct from binary Yjs blobs. When a client's cursor position changes, it sends a `type: "presence"` JSON message; the server broadcasts it to all other connected clients, which render a cursor indicator in the UI. This reuses the existing WebSocket connection without requiring a separate signaling channel. This helps enhance frontend usability by making collaboration state visible in real time, allowing clients to identify which user made each edit and how many collaborators are simultaneously active on the shared document (Figure 3).

4.2 WebSocket Server

The WebSocket server is implemented from scratch using cotamer's async TCP primitives, with no external WebSocket library. It implements the RFC 6455 essentials required to interoperate with a standard browser WebSocket client. The connection lifecycle begins with an HTTP upgrade handshake. The server reads the incoming HTTP request, extracts the `Sec-WebSocket-Key` header, concatenates it with the RFC 6455 GUID, computes a SHA-1 digest, and responds with the base64-encoded `Sec-WebSocket-Accept` header. Once the handshake completes, the connection is treated as a full-duplex binary channel.

Incoming frames are parsed according to the RFC 6455 framing format: the server reads the opcode, payload length, and the four-byte masking key that browsers are required to apply to all frames. Payload bytes are unmasked before being passed to the application layer. Control frames are handled inline: ping frames receive an immediate pong response, and close frames trigger a clean connection teardown.

Connected clients are tracked in a `ws_hub` that maintains the list of active file descriptors. Outbound committed blobs are sent as binary frames (opcode `0x2`) to every socket in the hub. Binary and text data frames from the browser are treated as opaque blobs for backend processing, with the exception of JSON presence messages which are handled separately before the blob is forwarded to the bridge.

4.3 Paxos Integration

The Paxos integration connects the WebSocket layer to the Multi-Paxos replication path through the `client_bridge`

abstraction defined in Section 3.4. The bridge holds two queues: `incoming_blobs` and `outgoing_blobs`, paired with cotamer events `incoming_ready` and `outgoing_ready` that allow the WebSocket and Paxos coroutines to signal each other without polling. An additional `committed_blob_log` vector holds the ordered history of all committed blobs for catch-up replay (Section 4.4).

When the server starts in `--websocket` mode, two coroutines are detached alongside the normal Paxos replica coroutines: `run_websocket_server()`, which handles browser connections, and `bridge_incoming_drainer()`, which moves blobs from `incoming_blobs` into the leader replica's `pending_bridge_blobs` queue. The drainer `co_awaits` on `incoming_ready`, wakes when the WebSocket server pushes a new blob, drains the queue, and appends each blob to the leader's pending queue before going back to sleep.

In the leader replica, pending bridge blobs are assigned new log slots as blob-bearing Paxos entries alongside normal key-value entries from the legacy client model. The legacy key-value implementation still remains in the implementation to test the Multi-Paxos implementation in isolation from the web-socket logic. The leader then runs standard Multi-Paxos phase 2 by broadcasting a `paxos_propose` to all replicas and waiting for acknowledgements from a quorum, finally marking an entry as decided. On commit, `apply_decided_prefix()` appends the blob to `committed_blob_log`, enqueues it on `outgoing_blobs`, and triggers `outgoing_ready`, waking the broadcast loop which sends the blob as a binary frame to all connected clients.

4.4 Late-Join Catch-Up

When a new browser client connects and completes the WebSocket handshake, the server executes a catch-up replay before admitting the client to the live broadcast hub. The implementation snapshots the current length of `committed_blob_log` at the moment the handshake completes, then iterates through the log from index zero, sending each blob as a binary frame. Because the Paxos leader may continue committing new blobs during the replay, appending to `committed_blob_log` beyond the snapshotted length, the loop does not stop at the snapshot boundary. Instead it continues until it reaches the current end of the log, at which point the client is inserted into the `ws_hub` and begins receiving live broadcasts.

This design ensures there is no gap between replayed history and the live stream. The client cannot miss a blob committed during replay because it is not added to the hub until the replay has caught up to the current log head. Once admitted, the client's document state is identical to that of any existing client that has applied the same prefix of the log. Because a context switch can only occur at a `co_await` point, the replay loop and the broadcast loop cannot interleave within a single frame send. This means the log snapshot and the subsequent replay are executed atomically with respect to other coroutines, eliminating the need for any explicit synchronization on `committed_blob_log`.

5 Evaluation

I evaluated the system on three metrics: write latency, failover delay, and correctness under concurrency.

5.1 Write Latency

To evaluate the latency overhead introduced by consensus replication, I measured end-to-end collaboration latency under four deployment configurations: a single-server baseline with no consensus replication, and Multi-Paxos clusters of 3, 5, and 7 replicas. The evaluation was performed using the full browser-based collaboration workload, executed against the real frontend, WebSocket layer, and Paxos backend.

The workload simulates two independent users by launching two isolated Chromium browser contexts connected to the same shared document. Each run performs a fixed sequence of collaborative operations: an initial document reset, an ordered baseline write, eight waves of concurrent edits from both users, followed by serialized tail appends from each client and a final convergence check. Each concurrent wave appends a 96-character token from both users simultaneously.

To evaluate the latency overhead introduced by consensus replication, I measured the time required for a single ordered edit to propagate from one browser client to another under different deployment configurations. Specifically, one client writes the string `"base|"` and the test measures the time until the second client observes the replicated update.

Configuration	Ordered Write Latency (ms)
No consensus	237.7
Paxos ($n = 3$)	991.7
Paxos ($n = 5$)	886.1
Paxos ($n = 7$)	910.8

Table 1: Latency of a single ordered edit becoming visible on another client.

The results show a clear latency increase once consensus replication is introduced. The no-consensus baseline completes the ordered write in approximately 238 ms because the server simply relays updates directly between connected clients, while the Paxos-backed configurations require roughly 0.9-1.0 s for the same operation with `link_delay` set to 150 ms to reflect real deployment environment. The largest latency jump occurs between the non-replicated baseline and the first replicated configuration, whereas increasing the replica count from 3 to 5 or 7 has relatively little additional impact since the overall protocol structure and number of consensus phases remain unchanged.

This reflects the fundamental tradeoff between latency and fault tolerance in replicated systems: Paxos replication increases write visibility latency because updates must be coordinated across a quorum before commitment, but in exchange the system gains resilience to replica failures while maintaining a globally consistent ordering of edits.

It is important to note, however, that these experiments were run entirely on a single machine using a simulated inter-replica

link delay rather than a real distributed deployment. As a result, while the measurements capture the coordination overhead imposed by Paxos, they may not fully reflect the behavior of production deployments as we will discuss in Section 6.

5.2 Failover Delay

To evaluate fault tolerance under server failure, I measured system recovery behavior after abruptly terminating the leader server process during an active collaborative editing session. The benchmark was executed against the same four deployment configurations used in the latency evaluation: a single-server no-consensus baseline and Multi-Paxos clusters of 3, 5, and 7 replicas tolerating one failure.

During the experiment, the browser continuously transmitted one Yjs binary blob every 100 ms over the existing WebSocket connection. The active server process was then terminated using `SIGKILL`, and the system was observed for an additional 35 s to determine whether replication recovered and broadcasts resumed.

Configuration	Downtime (ms)	Blobs Lost	Recovered
Baseline	∞	0	No
Paxos ($n = 3$)	3835.0	37	Yes
Paxos ($n = 5$)	6913.4	68	Yes
Paxos ($n = 7$)	6215.7	62	Yes

Table 2: Recovery behavior after abrupt leader failure.

The results demonstrate the fundamental availability advantage of replicated consensus over a single-server deployment. In the no-consensus baseline, killing the server permanently halted progress because there was no recovery path or replica redundancy. In contrast, all Paxos-backed configurations successfully recovered after leader failure, confirming that the failover and re-election mechanisms function correctly in the collaborative editing setting.

Recovery delays were on the order of several seconds, ranging from approximately 3.8 s to 6.9 s depending on configuration. During this interval, browser edits continued to be generated but could not be committed until a new leader was elected and resumed processing queued blobs. This resulted in a number of lost in-flight blobs during the outage window, with larger delays generally corresponding to more lost edits. Interestingly, recovery time did not increase monotonically with replica count: the 7-replica configuration recovered slightly faster than the 5-replica configuration in this run. This is realistic in distributed consensus systems, where election timing, scheduling jitter, and message ordering can introduce variability between trials even under identical configurations.

5.3 Correctness Under Concurrency

Finally, I evaluated whether the system preserves eventual consistency under high fan-out concurrent editing. This test stresses the full application stack by connecting many independent browser clients to the same shared document and issuing repeated waves of concurrent edits. Unlike the earlier latency benchmarks, which primarily measured ordered propagation delay between two users, this experiment focuses on whether all clients ultimately converge to the same document state under sustained concurrent load.

The stress benchmark was executed against a 3-replica Paxos deployment. The run used 25 browser clients, 5 concurrent edit waves, and 25 simultaneous appends per wave. After the document was reset to empty, all clients repeatedly appended to the shared document at approximately the same time before waiting for convergence. The test passed successfully: all 25 clients converged to an identical final document with length 16875 after approximately 38 seconds of execution.

Metric	Result
Clients	25
Concurrent waves	5
Concurrent appends per wave	25
Final document length	16875
Converged successfully	Yes
Runtime	38.4s

Table 3: Successful high-concurrency stress run.

However, increasing the workload further exposed important scalability limitations. When the same experiment was repeated with 10 waves instead of 5, some clients diverged and failed to converge to a single final document state. While the replicated Paxos log itself still imposes a globally ordered sequence of committed blobs, the overall system architecture introduces several practical bottlenecks under heavier sustained load.

The most likely limitation is the single-leader design of Multi-Paxos. In the current architecture, all browser edits are funneled through one leader replica, which must simultaneously receive WebSocket traffic, assign log slots, replicate blobs to a quorum, wait for acknowledgements, commit entries, and broadcast committed updates back to every connected client. With 25 clients issuing large bursts of concurrent appends, the leader and broadcast path can become saturated, creating backpressure and uneven update delivery across clients. We discuss possible mitigation strategies in Section 6.

Additionally, the frontend implementation itself may contribute to divergence under extreme concurrency. The editor uses a plain `<textarea>` synchronized with Yjs through manual diffing and DOM update propagation rather than a native CRDT-backed editor binding. Under sufficiently rapid concurrent updates, local textarea edits and remote Yjs updates may race with each other, causing some clients to compute updates from stale local state or temporarily overwrite remote changes before convergence completes.

6 Conclusion, Limitations, and Future Work

This paper presented a fault-tolerant collaborative text editor built on top of a Multi-Paxos replicated log and a Yjs CRDT frontend. By combining a globally ordered replication layer with deterministic CRDT updates, the system cleanly separates the problems of ordering and convergence while still achieving eventual consistency under concurrent editing. The evaluation demonstrated the expected tradeoffs between latency and fault tolerance: introducing Paxos replication increased write latency and introduced temporary downtime during failover, but enabled recovery from replica failures while preserving a globally consistent ordering of edits. Under moderate concurrent workloads, the system successfully converged across many simultaneous clients, showing that the overall architecture is capable of supporting collaborative editing at meaningful scale.

However, several limitations remain. First, all experiments were performed on a single machine using a simulated inter-replica network rather than a real distributed deployment. In production settings, replicas would likely execute on different physical machines or datacenters and communicate over real network connections rather than the in-process simulated network used in this implementation. Consequently, while the experiments accurately capture the coordination overhead imposed by consensus, the exact latency values may not directly generalize to production-scale deployments.

Second, the current system uses a single-leader Multi-Paxos architecture where all browser edits are funneled through one leader replica. Under heavier stress workloads, this leader became a scalability bottleneck because it was responsible for receiving client traffic, replicating blobs, committing entries, and broadcasting updates back to all connected clients simultaneously. This limitation became visible during the higher-intensity concurrency experiments where some clients diverged under sustained load.

Finally, the frontend implementation itself remains relatively lightweight and relies on synchronizing a plain `<textarea>` with Yjs through manual diffing and DOM updates. While sufficient for moderate workloads, this approach is more fragile than a fully native CRDT-backed editor integration and may contribute to race conditions under extreme concurrent editing pressure.

A promising direction for future work is to expose a WebSocket endpoint on every Paxos replica rather than routing all browser traffic through the leader. In such a design, clients could connect to any replica, potentially choosing the geographically closest one, while replicas internally forward edits into the Paxos replication path. This could reduce client-to-cluster latency, distribute frontend connection load across replicas, and alleviate pressure on the leader during periods of heavy concurrent editing. More broadly, future work could explore deploying the system across multiple real machines or datacenters, evaluating behavior under realistic WAN conditions, and integrating a richer CRDT-native editor frontend capable of handling substantially larger concurrent workloads.

References

[1] Tushar D. Chandra, Robert Griesemer, and Joshua Redstone. Paxos made live: An engineering perspective. *Proceedings of the Twenty-Sixth Annual ACM Symposium on Principles of Distributed Computing*, pages 398–407, 2007.

[2] Kevin Jahns. Yjs: A crdt framework for shared editing. `https://github.com/yjs/yjs`, 2020.

[3] Leslie Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.

[4] David A. Nichols, Pavel Curtis, Michael Dixon, and John Lamping. High-latency, low-bandwidth windowing in the jupiter collaboration system. In *Proceedings of the 8th Annual ACM Symposium on User Interface and Software Technology (UIST)*, pages 111–120. ACM, 1995.

[5] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. *Stabilization, Safety, and Security of Distributed Systems*, pages 386–400, 2011.